CTAB-GAN+: Enhancing Tabular Data Synthesis

Zilong Zhao*§, Aditya Kunar*§, Robert Birke†, Lydia Y. Chen*
*TU Delft, Netherlands {z.zhao-8, a.kunar, y.chen-10}@tudelft.nl
†ABB Research, Switzerland {robert.birke}@ch.abb.com

arXiv:2204.00401v1 [cs.LG] 1 Apr 2022

***Abstract*—While data sharing is crucial for knowledge development, privacy concerns and strict regulation (e.g., European General Data Protection Regulation (GDPR)) limit its full effectiveness. Synthetic tabular data emerges as alternative to enable data sharing while fulfilling regulatory and privacy constraints. State-of-the-art tabular data synthesizers draw methodologies from Generative Adversarial Networks (GAN). As GANs improve the synthesized data increasingly resemble the real data risking to leak privacy. Differential privacy (DP) provides theoretical guarantees on privacy loss but degrades data utility. Striking the best trade-off remains yet a challenging research question.**

We propose CTAB-GAN+ a novel conditional tabular GAN. CTAB-GAN+ improves upon state-of-the-art by (i) adding downstream losses to conditional GANs for higher utility synthetic data in both classification and regression domains; (ii) using Wasserstein loss with gradient penalty for better training convergence; (iii) introducing novel encoders targeting mixed continuous-categorical variables and variables with unbalanced or skewed data; and (iv) training with DP stochastic gradient descent to impose strict privacy guarantees. We extensively evaluate CTAB-GAN+ on data similarity and analysis utility against state-of-the-art tabular GANs. The results show that CTAB-GAN+ synthesizes privacy-preserving data with at least 48.16% higher utility across multiple datasets and learning tasks under different privacy budgets.

***Index Terms*—GAN; Data synthesis; Tabular data; Differential privacy; Imbalanced distribution**

I. Introduction

Many companies nowadays discover valuable insights from various internal and external data sources. However, the deep knowledge behind big data often violates personal privacy and leads to an unjustified analysis [22]. To prevent the abuse of data and the risks of privacy breaching, the European Commission introduced the European General Data Protection Regulation (GDPR) and enforced strict data protection measures. This however instills a new challenge in data-driven industries to look for new scientific solutions that can empower big discoveries while respecting the constraints of data privacy and governmental regulation.

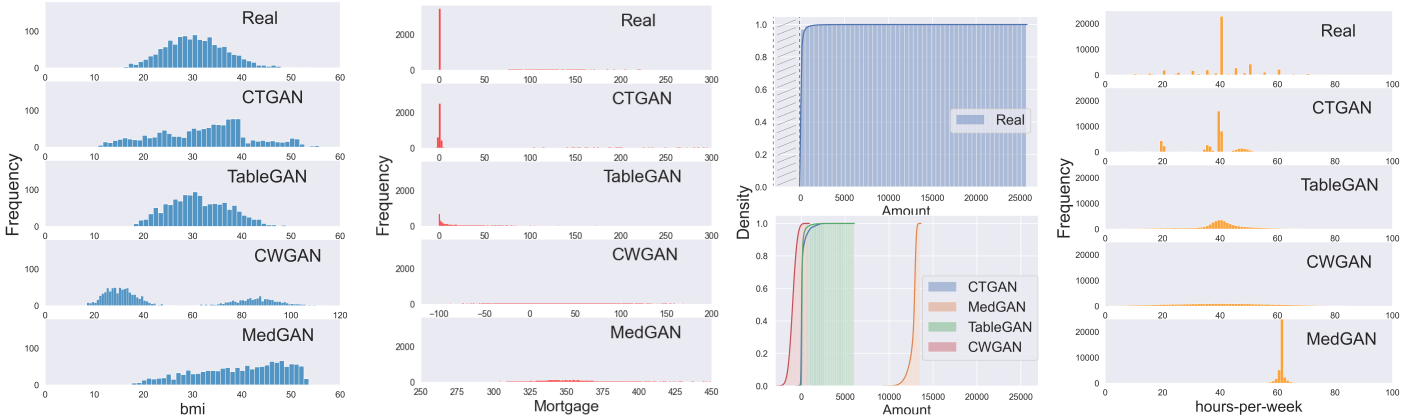
An emerging solution is to leverage synthetic data [21], which statistically resembles real data and can comply with GDPR due to its synthetic nature. Generative Adversarial Network (GAN) [12] is one of the emerging data synthesizing methodologies. Beyond its success in generating images [26], [21], [25], [33], [39] have recently applied GAN to generate tabular data. However, recent studies have shown that GANs may fall prey to membership inference attacks which greatly endanger the personal information present in the real training data [7], [27]. Therefore, it is imperative to safeguard the training of tabular GANs such that synthetic data can be generated without causing harm. To address these issues, prior work [16], [19], [28], [29] relies on differential privacy (DP) [9]. DP is a mathematical framework that provides theoretical guarantees bounding the statistical difference between any resulting ML model trained with or without a particular individual's information in the original training dataset. Typically, this can be achieved by injecting calibrated statistical noise while updating the parameters of a network during back-propagation, i.e., DP Stochastic Gradient Descent (DP-SGD) [1], [6], [32], or by injecting noise while aggregating teacher ensembles using the PATE framework [16], [24].

However, state-of-the-art (SOTA) tabular GAN algorithms only focus on two types of variables, namely continuous and categorical, overlooking an important class of mixed data type. In addition, it is unclear if existing solutions can efficiently handle highly imbalanced or skewed variables. Furthermore, most SOTA DP GANs are evaluated on images and their efficacy on tabular datasets needs to be verified. Existing DP GANs do not provide a well-defined consensus on which DP framework (i.e., DP-SGD or PATE) is optimal for training tabular GANs. Moreover, DP GAN algorithms such as [6] (GS-WGAN), [16] (PATE-GAN) change the original GAN structure from one discriminator to multiple discriminators, which increases the complexity of the algorithm. And [32] (DP-WGAN) and [28](RDP-GAN) use the weight clipping to bound gradients which introduces instability for GAN training.

In this paper, we extend CTAB-GAN [39] to a new algorithm CTAB-GAN+. The objectives of CTAB-GAN+ are two-folds: (1) further improve the synthetic data quality in terms of machine learning utility and statistical similarity; and (2) implement efficient DP into tabular GAN training to control its performance under different privacy budgets. To achieve the first goal, CTAB-GAN+ introduces a new feature encoder used with variables following single Gaussian distribution. Moreover, CTAB-GAN+ adopts the Wasserstein distance plus gradient penalty (hereinafter referred to as Was+GP) loss [13] to further enhance the stability and effectiveness of GAN training. Finally, CTAB-GAN+ adds a new auxiliary component to improve the synthesis performance for regression tasks. To achieve the second goal, CTAB-GAN+ uses DP-SGD algorithm to train a single instead of multiple discriminators as in PATE-GAN and GS-WGAN. This reduces the complexity of the algorithm. Additionally, CTAB-GAN+ reduces the privacy cost by accounting for sub-sampling [31] of smaller subsets from the full dataset used to train the discriminator.

We rigorously evaluate CTAB-GAN+ using two setups: (1)

§Equal contribution



(a) *bmi* in Insurance (b) *Mortgage* in Loan (c) *Amount* in Credit (d) *Hours-per-week* in Adult

Figure 1: Challenges of modeling industrial dataset using existing GAN-based table generator: (a) single Gaussian (b) mixed type, (c) long tail distribution, and (d) skewed data

without DP to generate data as realistic as possible; and (2) with DP under different privacy budgets to show the trade-off with data realism. Both setups rely on machine learning utility and statistical similarity of the synthetic data as evaluation metrics. Specifically, CTAB-GAN+ is tested on 7 widely used machine learning datasets: Adult, Covertype, Credit, Intrusion, Loan, Insurance and King against 7 SOTA GAN-based tabular data generation algorithms: CTGAN, TableGAN, CWGAN and MedGAN used in setup one, and PATE-GAN, DP-WGAN and GS-WGAN used in setup two. In setup one, CTAB-GAN+ outperforms all baselines on average by at least 41.2% on accuracy and 56.4% on AUC. in setup two under the same privacy budget (i.e., $\epsilon = 1$ and $\epsilon = 100$), CTAB-GAN+ outperforms all SOTA DP GANs on average by at least 48.16% on accuracy and 38.05% on F1-Score for the classification problem.

The main contributions of this study can be summarized as follows: (1) Novel conditional adversarial network which introduces a classifier/regressor providing additional supervision to improve the utility for ML applications. (2) Efficient modelling of continuous, categorical, and **mixed** variables via novel data encoding. (3) Improved GAN training using well-designed information, downstream, generator losses along with Was+GP to enhance stability and effectiveness. (4) Constructed a simpler and more stable DP GAN algorithm for tabular data to control its performance under different privacy budgets.

A. Motivation

We empirically demonstrate how the prior SOTA methods fall short in solving challenges in industrial data sets. The detailed experimental setup can be found in Sec. IV-A.

Single Gaussian variables. Single mode Gaussian distributions are very common. Fig. 1(a) shows the histogram of variable *bmi* (i.e., body mass index) in the Insurance dataset and synthetic data generated by 4 SOTA algorithms for this variable. The distribution of real data is close to a single mode Gaussian distribution. But except TableGAN, none of the SOTA algorithms can recover this distribution in their synthetic data. CTGAN uses variational Gaussian mixture (VGM) to model all continuous variables. However, VGM is a complicated method to deal with single mode Gaussian distributions as it initially approximates the distribution with multiple Gaussian mixtures by default. CWGAN and MedGAN use min-max normalization to scale the original data to [0, 1]. TableGAN also uses min-max normalization but scales the original data to [-1, 1] to better match the output of the generator using *tanh* as activation function. The reason that min-max normalization works for TableGAN but not MedGAN and CWGAN is because the training convergence for both algorithms is less stable than for TableGAN. However, since TableGAN applies min-max normalization on all variables, it suffers from a disadvantage modelling column with complex multi-modal Gaussian distributions.

Mixed data type variables. To the best of our knowledge, existing GAN-based tabular generators only consider table columns as either categorical or continuous. However, in reality, a variable can be a mix of these two types, and often variables have missing values. The *Mortgage* variable from the Loan dataset is a good example of mixed variable. Fig. 1(b) shows the distribution of the original and synthetic data generated by 4 SOTA algorithms for this variable. According to the data description, a loan holder can either have no mortgage (0 value) or a mortgage (any positive value). In appearance, this variable is not a categorical type due to the numeric nature of the data. So all 4 SOTA algorithms treat this variable as continuous type without capturing the special meaning of the value zero. Hence, all 4 algorithms generate a value around 0 instead of exact 0. And the negative values for Mortgage have no/wrong meaning in the real world.

Long tail distributions. Many real world data can have long tail distributions where most of the occurrences happen near the initial value of the distribution, and rare cases towards the end. Fig. 1(c) plots the cumulative frequency for the original (top) and synthetic (bottom) data generated by 4 SOTA algorithms for the *Amount* in the Credit dataset. This variable

represents the transaction amount when using credit cards. One can imagine that most transactions have small amounts, ranging from a few bucks to thousands of dollars. However, there definitely exists a very small number of transactions with large amounts. Note that for ease of comparison both plots use the same x-axis, but Real has no negative values. Real data clearly has 99% of occurrences happening at the start of the range, but the distribution extends until around 25000. In comparison none of the synthetic data generators is able to learn and imitate this behavior.

Skewed multi-mode continuous variables. The term *multi-mode* is extended from Variational Gaussian Mixtures (VGM). More details are given in Sec. III-C. The intuition behind using multiple modes can be easily captured from Fig. 1(d). The figure plots in each row the distribution of the working *Hours-per-week* variable from the Adult dataset. This is not a typical Gaussian distribution. There is an obvious peak at 40 hours but with several other lower peaks, e.g. at 50, 20 and 45. Also the number of people working 20 hours per week is higher than those working 10 or 30 hours per week. This behavior is difficult to capture for the SOTA data generators (see subsequent rows in Fig.1(d)). The closest results are obtained by CTGAN which uses Gaussian mixture estimation for continuous variables. However, CTGAN loses some modes compared to the original distribution.

The above examples show the shortcomings of current SOTA GAN-based tabular data generation algorithms and motivate the design of our proposed CTAB-GAN+.

II. Related Work

We divide the related work using GAN to generate tabular data into three: (i) based on GAN, (ii) based on conditional GAN, and (iii) based on DP GAN.

GAN-based generator. Several studies extend GAN to accommodate categorical variables by augmenting GAN architecture. MedGAN [8] combines an auto-encoder with a GAN. It can generate continuous or discrete variables, and has been applied to generate synthetic electronic health record (EHR) data. CrGAN-Cnet [21] uses GAN to conduct Airline Passenger Name Record Generation. It integrates the Cramér Distance [4] and Cross-Net architecture [30] into the algorithm. In addition to generating with continuous and categorical data types, CrGAN-Cnet can also handle missing value in the table by adding new variables. TableGAN [25] introduces information loss and a classifier into GAN framework. It specifically adopts Convolutional Neural Network (CNN) for generator, discriminator and classifier. Although aforementioned algorithms can generate tabular data, they cannot specify how to generate from a specific class for particular variable. For example, it is not possible to generate health record for users whose sex is female.

Conditional GAN-based generator. Due to the limitation of controlling generated data via GAN, Conditional GAN is increasingly used, and its conditional vector can be used to specify to generate a particular class of data. This feature is important when our available data is limited and highly skewed, and we need synthetic data of a specific class to re-balance the distribution. For instance, for preparing starting dataset of online learning scenarios [35], [37], [38]. CW-GAN [11] applies the Wasserstein distance [2] into the conditional GAN framework. It leverages the usage of conditional vector to oversample the minority class to address imbalanced tabular data generation. CTGAN [33] integrates PacGAN [18] structure in its discriminator and uses Generator loss and WGAN loss plus gradient penalty (GP) [13] to train a conditional GAN framework. It also adopts a strategy called training-by-sampling, which takes advantage of conditional vector, to deal with the imbalanced categorical variable problem.

CTAB-GAN+ not only focuses on modelling both continuous and categorical variables, but also covers the mixed data type (i.e., variables that contain both categorical and continuous values, or even missing values). We effectively combine the strengths of the prior art, such as Was+GP, classifier, information and generator losses along with an effective encoding. Furthermore, we proactively address the pain points of single Gaussian and long tail variable distributions and propose a new conditional vector structure to better deal with imbalanced datasets.

Differential Private Tabular GANs. To avoid leaking sensitive information on single individuals, previous studies explore multiple differential private learning techniques applied to GANs. Table I provides an overview. PATE-GAN [16] uses PATE [24] which relies on output sanitization by perturbing the output of an ensemble of teacher discriminators via Laplacian noise to train a student discriminator scoring the generated samples. One key limitation is that the student discriminator only sees synthetic data. Since this data is potentially unrealistic, the provided feedback can be unreliable. [32] (DP-WGAN), [6] (GS-WGAN) and [28](RDP-GAN) use differential private stochastic gradient descent (DP-SGD) coupled with the Wasserstein loss. Moreover, DP-WGAN uses a momentum accountant whereas GS-WGAN and RDP-GAN use a Rényi Differential Privacy (RDP) accountant. The Wasserstein loss is known to be more effective against mode-collapse compared to KL divergence [3]. The RDP accountant provides tighter bounds on the privacy costs improving the privacy-utility trade-off. To incorporate differential privacy guarantees and make the training compatible with the Wasserstein Loss, [28], [32] use weight clipping to enforce the Lipschitz constraint. The drawback is the need for careful tuning of the clipping parameter (see Sec. III-G). To overcome this issue, [6] enforces the Lipschitz constraint via a gradient penalty term as suggested by [14], but addresses only images which are a better fit for GANs and studies its efficacy only for training the generator network.

The proposed CTAB-GAN+ leverages RDP-based privacy accounting comparing to PATE used by PATE-GAN. Same as DP-WGAN, CTAB-GAN+ uses one discriminator instead of multiple ones trained by PATE-GAN and GS-WGAN. Since CTAB-GAN+ adopts Was+GP loss, it intrinsically constraints the gradient norm allowing to forgo the weight clipping used in DP-WGAN. This leads to a more stable training. In a nutshell, CTAB-GAN+ by training only one discriminator with Was+GP loss results in a more stable DP GAN algorithm compared to the SOTA algorithms.

Table I: Overview of DP GANs

Model	DP Algo	Loss	WC	DP Site	Single $\mathcal{D}$	Noise	Account.	Data Format
PATE-GAN	PATE	KL Diver.	No	$\mathcal{D}$	No	Lap.	PATE	Table
DP-WGAN	DP-SGD	Was.	Yes	$\mathcal{D}$	Yes	$\mathcal{N}$	Moment	Image & Table
GS-WGAN	DP-SGD	Was. + GP	No	$\mathcal{G}$	No	$\mathcal{N}$	RDP	Image
RDP-GAN	DP-SGD	Was.	Yes	$\mathcal{D}$	Yes	$\mathcal{N}$	RDP	Table
CTAB-GAN+	DP-SGD	Was. + GP	No	$\mathcal{D}$	Yes	$\mathcal{N}$	RDP	Table

* Was. (Wasserstein), WC (Weight Clipping), GP (Gradient Penalty), Lap. (Laplacian noise)
* $\mathcal{N}$ (Gaussian noise), $\mathcal{G}$ (Generator), $\mathcal{D}$ (Discriminator)

III. CTAB-GAN+

CTAB-GAN+ is a tabular data generator designed to overcome the challenges outlined in Sec. I-A. CTAB-GAN+ adopts a re-designed min-max scaler to normalize single Gaussian variable. We also propose a novel *Mixed-type Encoder* which can better represent mixed categorical-continuous variables as well as missing values. CTAB-GAN+ is based on a conditional GAN (CGAN) to efficiently treat minority classes, with the addition of Was+GP, downstream, information and generator losses ([13], [23], [25], [33]) to improve data quality and training stability. Moreover, we leverage a log-frequency sampler to overcome the mode collapse problem for imbalanced variables. Finally, differential private SGD training is implemented for the discriminator to achieve strict privacy guarantees.

A. Technical Background

1) Tabular GAN: GANs are a popular method to generate synthetic data first applied with great success to images [17] and later adapted to tabular data [34]. GANs leverage an adversarial game between a generator trying to synthesize realistic data and a discriminator trying to discern synthetic from real samples.

To address the problem of dataset imbalance, we leverage *conditional generator* and *training-by-sampling* methods from CTGAN. The idea behind this is to use an additional vector, termed conditional vector, to represent the classes of categorical variables. This vector is both fed to the generator and used to bound the sampling of the real training data to subsets satisfying the condition. We can leverage the condition to resample all classes giving higher chances to minority classes to train the model. To improve the stability of GAN training, CTAB-GAN+ adopts Was+GP [13] loss. Previous study WGAN [2] offers stability in training GAN. However, the use of gradient clipping leads to issues such as exploding and vanishing gradients. Comparing to WGAN [2], WGAN-GP replaces weight clipping with a constraint on the gradient norm of the discriminator to enforce Lipschitz continuity. This further stabilizes the training of the network and requires less hyper-parameter tuning. Another unique feature of WGAN-GP is that its discriminator updates 5 times per mini-batch data comparing to only 1 time of generator. This influences our differential privacy budget (see details in Sec. III-G). To enhance the generation quality, we incorporate three extra terms into the loss function of the generator: information [25], downstream (referred as classification loss in [23] for classification problems) and generator loss [33]. The information loss penalizes the discrepancy between statistics of the generated data and the real data. This helps to generate data which is statistically closer to the real one. The downstream loss requires adding to the GAN architecture an auxiliary classifier (or regressor) in parallel to the discriminator. For each synthesized value the classifier (or regressor) outputs a predicted value. The downstream loss quantifies the discrepancy between the synthesized and predicted values in the downstream analysis. This helps increase the semantic integrity of synthetic records. For instance, for a classification dataset, (sex=female, disease=prostate cancer) is not a semantically correct record as women do not have a prostate, and no such record should appear in the original data and is hence not learnt by the classifier. The generator loss measures the difference between the given conditions and the output classes of the generator. This loss helps the generator to learn to produce the exact same classes as the given conditions. Downstream loss is used by TableGAN but not by CTGAN, since CTGAN does not contain a classifier. Whereas, the generator loss is implemented by CTGAN but not by TableGAN, as TableGAN is not a conditional GAN. Both only treat classification problems.

To counter complex distributions in continuous variables we embrace the *Mode-Specific Normalization (MSN)* idea [33] which encodes each value as a value-mode pair stemming from the Gaussian mixture model.

2) Differential Privacy: DP is becoming the standard solution for privacy protection and has even been adopted by the US census department to bolster privacy of citizens [15]. DP protects against privacy attacks by minimizing the influence of any individual data point based on a given privacy budget. In this work, we leverage the Rényi Differential Privacy (RDP) [20] as it provides stricter bounds on the privacy budget. A randomized mechanism $\mathcal{M}$ is (λ, ϵ)-RDP with order λ, if $D_\lambda(\mathcal{M}(S)||\mathcal{M}(S')) = \frac{1}{\lambda-1} log\mathbb{E}_{x\sim\mathcal{M}(S)}\left[\left(\frac{P[\mathcal{M}(S)=x]}{P[\mathcal{M}(S')=x]}\right)\right]^{\lambda-1} \leq \epsilon$ holds for any adjacent datasets S and S', where $D_\lambda(P||Q) = \frac{1}{\lambda-1} log\mathbb{E}_{x\sim Q}[(P(x)/Q(x))^\lambda]$ represents the Rényi divergence. In addition, a (λ, ϵ)-RDP mechanism $\mathcal{M}$ can be expressed as:

$$(\epsilon + \frac{log1/\delta}{\lambda - 1}, \delta)\text{-DP}. \tag{1}$$

For the purpose of this work $\mathcal{M}$ corresponds to a tabular GAN model with privacy budget (λ, ϵ).

RDP is a strictly stronger privacy definition than DP as it provides tighter bounds for tracking the cumulative privacy loss over a sequence of mechanisms via the Composition theorem [20]. Let $\circ$ denote the composition operator. For $\mathcal{M}_1$,...,$\mathcal{M}_k$ all being (λ, ϵ_i)-RDP, the composition $\mathcal{M}_1 \circ ... \circ \mathcal{M}_k$ is

$$(\lambda, \sum_i \epsilon_i)\text{-RDP} \tag{2}$$

Additionally, for a Gaussian Mechanism [10] $\mathcal{M}_\sigma$ parameterized by σ as:

$$\mathcal{M}_\sigma(x) = f(x) + \mathcal{N}(0, \sigma^2 I) \tag{3}$$

where f denotes an arbitrary function with sensitivity $\Delta_{2f} = \max_{S,S'} ||f(S) - f(S')||_2$ over all adjacent datasets S and S', and $\mathcal{N}$ represents a Gaussian distribution with zero mean and covariance $\sigma^2 I$ (where I is the identity matrix), $\mathcal{M}_\sigma$ satisfies $(\lambda, \frac{\lambda \Delta_{2f}^2}{2\sigma^2})$-RDP [20].

Lastly, two more theorems are key to this work. The post processing theorem [10] states that if $\mathcal{M}$ satisfies (ϵ, δ)-DP,

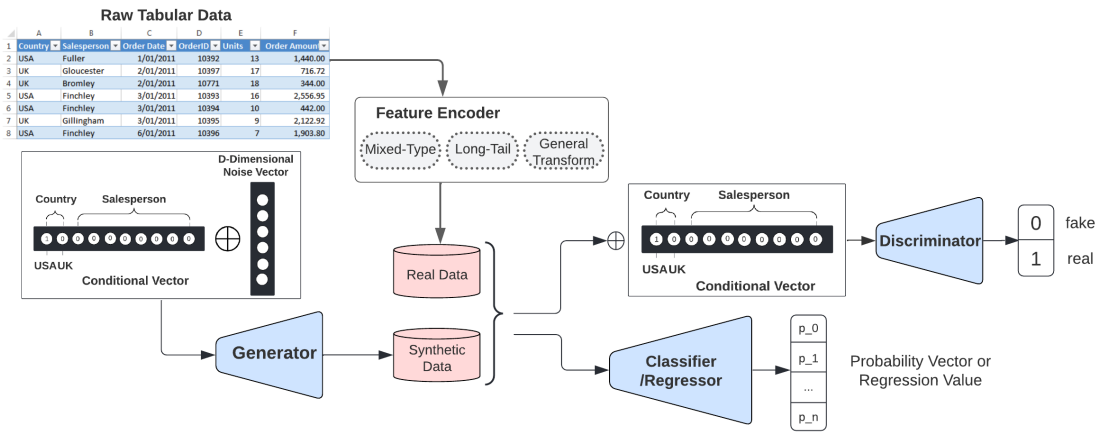


Figure 2: Synthetic Tabular Data Generation via CTAB-GAN+

$F \circ \mathcal{M}$ will satisfy (ϵ, δ)-DP, where F can be any arbitrary randomized function. Hence, it suffices to train one of the two networks in the GAN architecture with DP guarantees to ensure that the overall GAN is compatible with differential privacy. RDP for subsampled mechanisms [31] computes the reduction in privacy budget when sub-sampling private data. Formally, let $\mathcal{X}$ be a dataset with n data points and **subsample** return $m \leq n$ subsamples without replacement from $\mathcal{X}$ (subsampling rate $\gamma = m/n$). For all integers $\lambda \geq 2$, if a randomized mechanism $\mathcal{M}$ is $(\lambda, \epsilon(\lambda))$-RDP, then $\mathcal{M} \circ$**subsample** is

$$(\lambda, \epsilon'(\lambda))\text{-RDP} \tag{4}$$

where $\epsilon'(\lambda) \leq \frac{1}{\lambda-1} log(1 + \gamma^2 \binom{\lambda}{2} \min\left\{4(e^{\epsilon(2)} - 1), e^{\epsilon(2)} \min\{2, (e^{\epsilon(\infty)} - 1)^2\}\right\} + \sum_{j=3}^{\lambda} \gamma^j \binom{\lambda}{j} e^{(j-1)\epsilon(j)} \min\{2, (e^{\epsilon(\infty)-1)^j})$

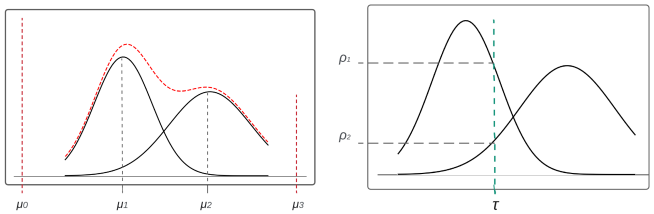
B. Architecture of CTAB-GAN+

The structure of CTAB-GAN+ is shown in Fig. 2. It comprises three blocks: Generator $\mathcal{G}$, Discriminator $\mathcal{D}$ and an auxiliary component (either a classifier or a regressor) $\mathcal{C}$. Since our algorithm is based on conditional GAN, the generator requires a noise vector plus a conditional vector. Details on the conditional vector are given in Sec. III-D. Before feeding data to $\mathcal{D}$ and $\mathcal{C}$, variables are encoded via different feature encoders depending on the variable type and characteristics. Details of the used encoders are provided in Sec. III-C, III-E and III-F.

GANs are trained via a zero-sum min-max game where the discriminator tries to maximize the objective, while the generator tries to minimize it. The game can be seen as a mentor ($\mathcal{D}$) providing feedback to a student ($\mathcal{G}$) on the quality of his work. Here, we introduce additional feedback for $\mathcal{G}$ based on the information loss, downstream loss and generator loss. The information loss matches the first-order (i.e., mean) and second-order (i.e., standard deviation) statistics of synthesized and real records. This leads the synthetic records to have the same statistical characteristics as the real records. The downstream loss equates the correlation between target variable and the other variable values. This helps to check the semantic integrity, and penalizes synthesized records where the combination of values are semantically incorrect. Finally, the generator loss is the cross-entropy between the given conditional vector and the generated output classes. It enforces the conditional generator to produce the same classes as the given conditional vector. These three losses add to the default loss term (i.e., Was+GP) of $\mathcal{G}$ during training. $\mathcal{G}$ and $\mathcal{D}$ are implemented using CNNs with the same structure as in [25]. CNNs are good at capturing the relation between pixels within an image, which in our case, can help to increase the semantic integrity of synthetic data. To process row records stored as vectors with CNN, we wrap the row data into the closest square matrix dimensions, i.e. $d \times d$ where d is the ceiled square root of the row data dimensionality and pad missing values with zeros. $\mathcal{C}$ uses a multi-layer-perceptron (MLP) with four 256-neuron hidden layers. The classifier is trained on the original data to better interpret the semantic integrity. Hence synthetic data are reverse transformed from their matrix encoding to vector (details in Sec. III-C). Real data is encoded (details in Sec. III-C and III-F) before being used as input for $\mathcal{C}$ to create the class label predictions.

Let f_x and $f_{\mathcal{G}(z)}$ denote the features fed into the softmax layer of $\mathcal{D}$ for a real sample x and a sample generated from latent value z, respectively. The **information loss** for $\mathcal{G}$ is expressed as $\mathcal{L}^G_{info} = ||\mathbb{E}[f_x]_{x \sim p_{data}(x)} - \mathbb{E}[f_{\mathcal{G}(z)}]_{z \sim p(z)}||_2 + ||\mathbb{SD}[f_x]_{x \sim p_{data}(x)} - \mathbb{SD}[f_{\mathcal{G}(z)}]_{z \sim p(z)}||_2$ where $p_{data}(x)$ and $p(z)$ denote prior distributions for real data and latent variable, $\mathbb{E}$ and $\mathbb{SD}$ denote the mean and standard deviations of the features, respectively. The **downstream loss** is given by $\mathcal{L}^G_{dstream} = \mathbb{E}[|l(G(z)) - \mathcal{C}(fe(G(z)))|]_{z \sim p(z)}$ where $l(.)$ returns the target variable and $fe(.)$ returns the input features of a given row x. Finally, the **generator loss** is given by $\mathcal{L}^G_{generator} = H(m_i, \hat{m}_i)$ where m_i and $\hat{m}_i$ are the given and generated conditional vector bits corresponding to column i and $H(.)$ is the cross-entropy loss. Columns are selected using the training-by-sampling procedure (see Sec. III-D for details).

Let $\mathcal{L}^D_{default}$ and $\mathcal{L}^G_{default}$ denote the GAN loss of discriminator and generator from Was+GP where its unique objective



(a) Mixed type variable distribution with VGM (b) Mode selection of single value in continuous variable

Figure 3: Encoding for mix data type variable

function of discriminator is defined as follows:

$$L = \underbrace{\mathop{\mathbb{E}}_{\tilde{x}\sim\mathbb{P}_g}[D(\tilde{x})] - \mathop{\mathbb{E}}_{x\sim\mathbb{P}_r}[D(x)]}_{\text{original discriminator loss}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{x}\sim\mathbb{P}_{\hat{x}}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1])^2}_{\text{gradient penalty}}$$

where $\mathbb{P}_{\hat{x}}$ is defined as sampling uniformly along straight lines between pairs of points sampled from the real data distribution $\mathbb{P}_r$ and the generator distribution $\mathbb{P}_g$. For $\mathcal{G}$ the complete training objective is $\mathcal{L}^G = \mathcal{L}^G_{default} + \mathcal{L}^G_{info} + \mathcal{L}^G_{dstream} + \mathcal{L}^G_{generator}$. The training objective for $\mathcal{D}$ is unchanged. Finally, the loss to train the auxiliary $\mathcal{C}$ is similar to the downstream loss of the generator, i.e. $\mathcal{L}^C_{dstream} = \mathbb{E}[|l(x) - \mathcal{C}(fe(x))|]_{x\sim p_{data}(x)}$.

C. Mixed-type Encoder

The tabular data is encoded variable by variable. We distinguish three types of variables: categorical, continuous and mixed. We define variables as mixed if they contain both categorical and continuous values or continuous values with missing values. We propose the new Mixed-type Encoder to deal with such variables. With this encoder, values of mixed variables are seen as concatenated value-mode pairs. We illustrate the encoding via the exemplary distribution of a mixed variable shown in red in Fig. 3(a). One can see that values can either be exactly μ_0 or μ_3 (the categorical part) or distributed around two peaks in μ_1 and μ_2 (the continuous part). We treat the continuous part by adapting the *Mode-Specific Normalization* (MSN) idea from [33] in using a variational Gaussian mixture model (VGM) [5] to estimate the number of modes k, e.g. $k = 2$ in our example, and fit a Gaussian mixture. The learned Gaussian mixture is $\mathbb{P} = \sum_{k=1}^{2} \omega_k \mathcal{N}(\mu_k, \sigma_k)$, where $\mathcal{N}$ is the normal distribution and ω_k, μ_k and σ_k are the weight, mean and standard deviation of each mode, respectively.

To encode values in the continuous region of the variable distribution, we associate and normalize each value with the mode having the highest probability (see Fig. 3(b)). Given ρ_1 and ρ_2 being the probability density from the two modes in correspondence of the variable value τ to encode, we select the mode with the highest probability. In our example ρ_1 is higher and we use mode 1 to normalize τ. The normalized value α is: $\alpha = \frac{\tau - \mu_1}{4\sigma_1}$. Moreover we keep track of the mode β used to encode τ via one-hot encoding, e.g. $\beta = [0, 1, 0, 0]$ in our example. The final encoding is giving by the concatenation

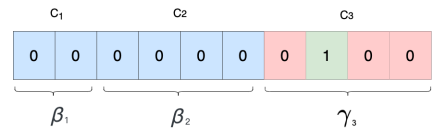


Figure 4: Conditional vector: example selects class 2 from third variable out of three

of α and β: $\alpha \bigoplus \beta$ where $\bigoplus$ is the vector concatenation operator. The categorical part (e.g., μ_0 or μ_3 in Fig. 3(a)) is treated similarly, except α is directly set to 0. Because the category is determined only by one-hot encoding part. For example, for a value in μ_3, the final encoding is given by $0 \bigoplus [0, 0, 0, 1]$. Categorical variables use the same encoding as the continuous intervals of mixed variables. Categorical variables are encoded via a one-hot vector γ. Missing values are treated as a separate unique class and we add an extra bit to the one-hot vector for it. A row with $[1, \ldots, N]$ variables is encoded by concatenation of the encoding of all variable values, i.e. either $(\alpha \bigoplus \beta)$ for continuous and mixed variables or γ for categorical variables. Having n continuous/mixed variables and m categorical variables ($n + m = N$) the final encoding is:

$$\bigoplus_{i=1}^{n} \alpha_i \bigoplus \beta_i \bigoplus_{j=n+1}^{N} \gamma_j \tag{5}$$

D. Counter Imbalanced Training Datasets

In CTAB-GAN+, we use conditional GAN to counter imbalanced training datasets using training-by-sampling [33], but extended to include the modes of continuous and mixed columns. When we sample real data, we use the conditional vector to filter and rebalance the training data. The conditional vector $\mathcal{V}$ is a bit vector given by the concatenation of all mode one-hot encodings β (for continuous and mixed variables) and all class one-hot encodings γ (for categorical variables) for all variables present in Eq. (5). Each conditional vector specifies a single mode or a class. More in detail, $\mathcal{V}$ is a zero vector with a single one in correspondence to the selected variable with selected mode/class. Fig. 4 shows an example with three variables, one continuous (C_1), one mixed (C_2) and one categorical (C_3), with class 2 selected on C_3.

To rebalance the dataset, each time we need a conditional vector during training, we first randomly choose a variable with uniform probability. Then we calculate the probability distribution of each mode (or class for categorical variables) in that variable using frequency as proxy and sample a mode based on the logarithm of its probability. Using the log probability instead of the original frequency gives minority modes/classes higher chances to appear during training. This helps to alleviate the collapse issue for rare modes/classes. Extending the conditional vector to include the continuous and mixed variables helps to deal with imbalance in the frequency of modes used to represent them. Moreover, since generator is conditioned on all data-types during training, this enhances the learned correlation between all variables.

E. General Transform

CTAB-GAN originally adopts the mode-specific-normalization (MSN) from CTGAN to encode all continuous variables. MSN uses VGM to estimate the distribution of continuous variables. Fig. 1(a) shows that VGM is not suitable for simple distributions such as single Gaussian. Another problem is the dimensionality explosion caused by using one-hot-encoding for categorical variables with a high number of categories. To counter both problems we propose the general transform (GT). GT is an effective approach to minimize the complexity of our algorithm.

The main idea of GT is to encode columns in the range of $(-1, 1)$. This makes the encoding directly compatible with the output range of the generator using *tanh* activation function. This is achieved via a shifted and scaled min-max normalization. Mathematically, given a data point x_i of a continuous variable x, the transformed value, $x_i^t = 2 * \frac{x_i - min(x)}{max(x) - min(x)} - 1$ where $min(x)$ and $max(x)$ represents the minimum and maximum values of the continuous variable. Inversely an encoded or generated value x_i^t may be reverse transformed as $X_i = (max(x) - min(x)) * \frac{X_i^t + 1}{2} + min(x)$. Continuous variable can be directly treated with the above formulas for normalization and denormalization. Categorical variables are first encoded using integers before using the above normalization and rounded to integers after using the above denormalization.

A similar transform was first introduced by TableGAN, but it applies this transformation on all variables. This choice is not optimal. From our experiments, we find that this technique only works well for continuous columns with simple distributions such as a single-mode Gaussian and does not cater to more complex distributions. By default, CTAB-GAN+ deals with continuous variable with MSN and only selectively uses GT for processing single-mode Gaussian variables. Similarly, categorical columns should prefer MSN as encoding rather than GT. Using GT loses the mode indicator, i.e. β_1 in Fig. 4, from the conditional vector forgoing the ability to enhance the correlation between variables for specific categories. Moreover, using integers instead of one-hot vectors can impose artificial distances between the different categories which do not reflect the reality. Therefore, we recommend to use GT for categorical variables only if the categorical variables contain so many categories that the available machines can not train with the encoded data.

F. Treat Long Tails

We encode continuous values using variational Gaussian mixtures to treat multi-mode data distributions (details in Sec. III-C). However, Gaussian mixtures can not deal with all types of data distribution, notable distributions with long tail where few rare points are far from the bulk of the data. VGM has difficulty to encode the values towards the tail. To counter this issue we pre-process variables with long tail distributions with a logarithm transformation. For such a variable having values with lower bound l, we replace each value τ with compressed τ^c:

$$\tau^c = \left\{ \begin{array}{ll} \log(\tau) & \text{if } l > 0 \\ \log(\tau - l + \epsilon) & \text{if } l \leqslant 0, \text{ where } \epsilon > 0 \end{array} \right\} \tag{6}$$

The log-transform allows to compress and reduce the distance between the tail and bulk data making it easier for VGM to encode all values, including tail ones. We show the effectiveness of this simple yet performant method in Sec. IV-E.

G. Differential Privacy

DP-SGD [1] is the central framework to provide DP guarantees in this work. DP-SGD uses noisy stochastic gradient descent to limit the influence of individual training samples x_i. After computing the gradient $g(x_i)$, the gradient is clipped based on a clipping parameter C and its L2 norm $\bar{g}(x_i) \leftarrow g(x_i)/\max(1, \frac{||g(x_i)||_2}{C})$, and Gaussian noise is added $\tilde{g}(x_i) \leftarrow \bar{g}(x_i) + \mathcal{N}(0, \sigma^2 C^2 I))$. $\tilde{g}$ is then used in place of g to update the network parameters as in traditional SGD.

One of the biggest challenges with DP-SGD is tuning the clipping parameter C since clipping greatly degrades the information stored in the original gradients [6]. Choosing an optimal clipping value that does not significantly impact utility is crucial. However, tuning the clipping parameter is laborious as the optimal value fluctuates depending on network hyperparameters (i.e. model architecture, learning rate) [1]. To avoid an intensive hyper-parameter search, [6] proposes to use the Wasserstein loss with a gradient penalty term. This term ensures that the discriminator generates bounded gradient norms which are close to 1 under real and generated distributions. Therefore, an optimal clipping threshold of $C = 1$ is obtained implicitly.

CTAB-GAN+ trains the discriminator using differential private-SGD where the number of training iterations is determined based on the total privacy budget (ϵ,δ). Thus, to compute the number of iterations, the privacy budget spent for every iteration must be bounded and accumulated. For this purpose we use the subsampled RDP analytical moments accountant technique.

Corollary 1. *Each discriminator update satisfies $(\lambda, 2B\lambda/\sigma^2)$-RDP where B is the batch size.*

Proof 1. *Let $f = clip(\bar{g}_D, C)$ be the clipped gradient of the discriminator before adding noise. The sensitivity is derived via the triangle inequality:*

$$\Delta_2 f = \max_{S,S'} ||f(S) - f(S')||_2 \leq 2C \tag{7}$$

Since $C = 1$ as a consequence of the Wasserstein loss with gradient penalty, and by using (3)*, the gaussian mechanism used within the DP-SGD procedure denoted as $\mathcal{M}_\sigma$ parameterized by noise scale σ may be represented as being $(\lambda, 2\lambda/\sigma^2)$-RDP.*

Furthermore, each discriminator update for a batch of real data points $\{x_i, .., x_B\}$ can be represented as

$$\tilde{g}_D = \frac{1}{B} \sum_{i=1}^{B} \mathcal{M}_\sigma(\nabla_{\theta_D} \mathcal{L}_D(\theta_D, x_i)) \tag{8}$$

where $\tilde{g}_D$ and θ_D represent the perturbed gradients and the weights of the discriminator network, respectively. This may be regarded as a composition of B Gaussian mechanisms and treated via (2)*. The privacy cost for a single gradient update step for the discriminator can be expressed as* $(\lambda, \sum_{i=1}^{B} 2\lambda/\sigma^2)$ *or equivalently* $(\lambda, 2B\lambda/\sigma^2)$.

Note that $\mathcal{M}_\sigma$ is only applied for those gradients that are computed with respect to the real training dataset [1], [36]. Hence, the gradients computed with respect to the synthetic data and the gradient penalty term are left undisturbed. Next, to further amplify the privacy protection of the discriminator, we rely on (4) with subsampling rate $\gamma = B/N$ where B is the batch size and N is the size of the training dataset. Intuitively, subsampling adds another layer of randomness and enhances privacy by decreasing the chances of leaking information about particular individuals who are not included in any given subsample of the dataset.

Lastly, it is worth mentioning that the Was+GP training objective has one major pitfall with respect to the privacy cost. This is because, it encourages the use of a stronger discriminator network to provide more meaningful gradient updates to the generator. This requires performing multiple updates to the discriminator for each corresponding update to the generator leading to a faster consumption of the overall privacy budget.

IV. Experimental Analysis for Data utility

To show the efficacy of the proposed CTAB-GAN+, we select seven commonly used machine learning datasets, and compare with four SOTA GAN based tabular data generators and CTAB-GAN. We evaluate the effectiveness of CTAB-GAN+ in terms of the resulting ML utility, statistical similarity to the real data. Moreover, we provide ablation analyses to highlight the efficacy of the unique components of CTAB-GAN+.

A. Experimental Setup

Datasets. Our algorithm is tested on seven commonly used machine learning datasets. Three of them **Adult**, **Covertype** and **Intrusion** are from the UCI machine learning repository[1]. **Credit** and **Loan** are from Kaggle[2]. The above five tabular datasets are used for classification tasks using as target a categorical variable. To consider also regression tasks we use two more datasets, **Insurance** and **King** from Kaggle[3] where the target variable is continuous.

Due to computing resource limitations, 50K rows of data are sampled randomly in a stratified manner with respect to the target variable for the Covertype, Credit and Intrusion datasets. The Adult, Loan, Insurance and King datasets are taken in their entirety. The details of each dataset are shown in Tab. II. We assume that the data type of each variable is known before training. [33] holds the same assumption.

[1]http://archive.ics.uci.edu/ml/datasets

[2]https://www.kaggle.com/{mlg-ulb/creditcardfraud,itsmesunil/bank-loan-modelling}

[3]https://www.kaggle.com/{mirichoi0218/insurance,harlfoxem/housesalesprediction}

Baselines. Our CTAB-GAN+ is compared with CTAB-GAN and 4 other SOTA GAN-based tabular data generators: CTGAN, TableGAN, CWGAN and MedGAN. To have a fair comparison, all algorithms are coded using Pytorch, with the generator and discriminator structures matching the descriptions provided in their respective papers. For Gaussian mixture estimation of continuous variables, we use the same settings as the evaluation of CTGAN, i.e. 10 modes. All algorithms are trained for 150 epochs for Adult, Covertype, Credit and Intrusion datasets, whereas the algorithms are trained for 300 epochs on Loan, Insurance and King datasets. The reason is these three datasets are smaller than the others and require more epochs to converge. Lastly, each experiment is repeated 3 times.

Environment. Experiments are run under Ubuntu 20.04 on a machine equipped with 32 GB memory, a GeForce RTX 2080 Ti GPU and a 10-core Intel i9 CPU.

B. Evaluation Metrics

The evaluation is conducted on two dimensions: (1) machine learning (ML) utility, and (2) statistical similarity. They measure if the synthetic data can be used as a good proxy of the original data.

1) Machine Learning Utility: The ML utility of classification and regression tasks is quantified differently. For classification, we quantify the ML utility via the performance, i.e, accuracy, F1-score and AUC, achieved by 5 widely used machine learning algorithms on real versus synthetic data: decision tree classifier, linear support-vector-machine (SVM), random forest classifier, multinomial logistic regression and MLP. Fig. 5 shows the evaluation process for classification datasets. The training dataset and synthetic dataset are of the same size. The aim is to show the difference in ML utility when a ML model is trained on synthetic vs real data. We use different classification performance metrics. Accuracy is the most commonly used, but does not cope well with imbalanced target variables. F1-score and AUC are more stable metrics for such cases. AUC ranges from 0 to 1. For regression tasks, we quantify the ML utility in a similar manner but using 4 common regression algorithms – linear regression, ridge regression, lasso regression and Bayesian ridge regression – and 3 regression metrics – mean absolute percentage error (MAPE), explained variance score (EVS) and R^2 score. All algorithms are implemented using scikit-learn 0.24.2 with default parameters except max-depth 28 for decision tree and random forest, and 128 neurons for MLP. For a fair comparison, hyper-parameters are fixed across all datasets. Due to this our results can slightly differ from [33] where the authors use different ML models and hyper-parameters for different datasets.

2) Statistical Similarity: Three metrics are used to quantify the statistical similarity between real and synthetic data.

Jensen-Shannon divergence (JSD). The JSD provides a measure to quantify the difference between the probability mass distributions of individual categorical variables belonging to the real and synthetic datasets, respectively. Moreover, this metric is bounded between 0 and 1 and is symmetric allowing for an easy interpretation of results.

Table II: Description of Datasets

Dataset	Problem	Train/Test Split	Target variable	Continuous	Binary	Multi-class	Mixed-type	Long-tail	General Transform
Adult	Classification	39k/9k	'income'	3	2	7	2	0	1
Covertype	Classification	45k/5k	'Cover_Type'	10	44	1	0	0	47
Credit	Classification	40k/10k	'Class'	30	1	0	0	1	29
Intrusion	Classification	45k/5k	'Class'	22	6	14	0	2	6
Loan	Classification	4k/1k	'PersonalLoan'	5	5	2	1	0	9
Insurance	Regression	1k/300	'charges'	13	3	2	2	0	1
King	Regression	17.3k/4.3k	'price'	1	6	2	0	0	6

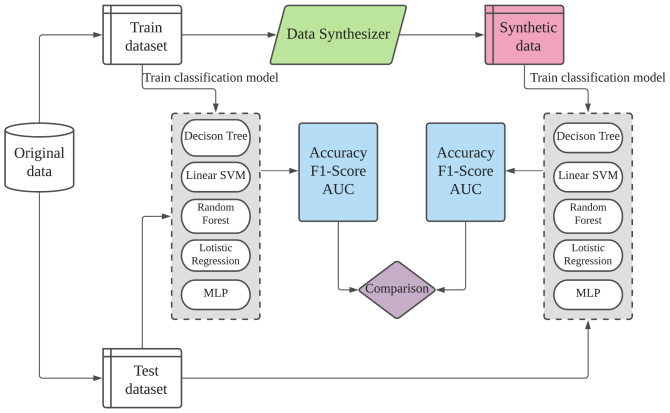


Figure 5: Evaluation flows for ML utility of Classification

Wasserstein distance (WD). In a similar vein, the Wasserstein distance is used to capture how well the distributions of individual continuous/mixed variables are emulated by synthetically produced datasets in correspondence to real datasets. We use WD because we found that the JSD metric was numerically unstable for evaluating the quality of continuous variables, especially when there is no overlap between the synthetic and original dataset. Hence, we resorted to utilize the more stable Wasserstein distance.

Difference in pair-wise correlation (Diff. Corr.). To evaluate how well feature interactions are preserved in the synthetic datasets, we first compute the pair-wise correlation matrix for the columns within real and synthetic datasets individually. Pearson correlation coefficient is used between any two continuous variables. It ranges between $[-1, +1]$. Similarly, the Theil uncertainty coefficient is used to measure the correlation between any two categorical features. It ranges between $[0, 1]$. And the correlation ratio between categorical and continuous variables is used. It also ranges between $[0, 1]$. Note that the dython[4] library is used to compute these metrics. Finally, the difference between pair-wise correlation matrices for real and synthetic datasets is computed.

C. Results Analysis

We first discuss the results in ML utility before addressing stochastic similarity.

ML Utility. Tab. III shows the results for the classification datasets. A better synthetic dataset is expected to have small differences in d ML utility for classification tasks trained on real and synthetic data. It can be seen that CTAB-GAN+ outperforms all other SOTA methods and CTAB-GAN in all

[4]http://shakedzy.xyz/dython/modules/nominal/#compute_associations

Table III: Difference of ML Utility and Statistical Similarity for Classification between original and synthetic data, averaged on five datasets

Method	ML Utility Difference			Statistical Similarity Difference		
	Accuracy	F1-score	AUC	Avg JSD	Avg WD	Diff. Corr.
CTAB-GAN+	**5.23%**	**0.090**	**0.041**	**0.039**	**484**	**2.03**
CTAB-GAN	8.90%	0.107	0.094	0.062	1197	2.09
CTGAN	21.51%	0.274	0.253	0.070	1769	2.73
TableGAN	11.40%	0.130	0.169	0.080	2117	2.30
MedGAN	14.11%	0.282	0.285	0.214	46257	5.48
CW-GAN	20.06%	0.354	0.299	0.132	238155	5.82

Table IV: Difference of ML Utility and Statistical Similarity for Regression between original and synthetic data, averaged on two datasets

Method	ML Utility Difference			Statistical Similarity Difference		
	MAPE	EVS	R^2	Avg JSD	Avg WD	Diff. Corr.
CTAB-GAN+	**0.04**	**0.03**	**0.04**	**0.040**	**856**	**0.65**
CTAB-GAN	0.06	0.05	0.06	0.119	3396	1.23
CTGAN	0.87	0.59	0.71	0.139	3030	2.60
TableGAN	0.34	0.43	0.48	0.317	2366	2.26
MedGAN	5.98	0.65	27.47	0.398	170307	7.27
CW-GAN	0.64	0.72	2.40	0.43	9.96E6	9.59

the metrics. CTAB-GAN+ decreases the AUC difference from 0.094 (best baseline) to 0.041 (56.4% reduction), and the difference in accuracy from 8.9% (best baseline) to 5.23% (41.2% reduction). The improvement over CTAB-GAN shows that general transform and Was+GP loss indeed help enhance the feature representation and GAN training. Tab. IV shows the results for the regression datasets. The result of CTAB-GAN and CTAB-GAN+ are far better than all other baselines. This shows the effectiveness of the feature engineering. Additionally, as CTAB-GAN+ adds the auxiliary regressor which explicitly enhances the regression analysis, the overall downstream performance of CTAB-GAN+ is better than CTAB-GAN. We note that CTAB-GAN uses auxiliary classification loss for the classification analysis and disables it for the regression analysis.

Statistical similarity. Statistical similarity results for the classification datasets are reported in Tab. III and for regression datasets in Tab. IV. CTAB-GAN+ stands out again across all baselines in both groups of datasets. For classification datasets, CTAB-GAN+ outperforms CTAB-GAN, CTGAN and TableGAN by 37.1%, 44.3% and 51.3% in average JSD. This is due to the use of the conditional vector, the log-frequency sampling and the extra losses, which work well for both balanced and imbalanced distributions. For continuous variables (i.e. average WD), the average WD column shows some extreme numbers such as 46257 and 238155 comparing to 484 of CTAB-GAN+. The reason is that these algorithms generate extremely large values for long tail variables. Comparing to CTAB-GAN, the significant improvement comes from the

use of general transform to model continuous columns with simple distributions which originally used MSN under CTAB-GAN and CTGAN. For regression datasets, CTAB-GAN+ outperforms CTAB-GAN by 63.4% and 74.5% in average JSD and average WD, respectively. Besides JSD and WD, the synthetic regression datasets maintain much better correlation than all the comparisons. This result confirms the efficacy of the usage of regressor.

Table V: Ablation Analysis For CTAB-GAN (F1. diff.)

Dataset	CTAB-GAN	w/o $\mathcal{C}$	w/o I. Loss	w/o MSN	w/o LT
Adult	0.704	-0.01	-0.037	-0.05	-
Covertype	0.532	-0.018	-0.184	-0.118	-
Credit	0.710	+0.011	-0.177	+0.06	+0.001
Intrusion	0.842	-0.031	-0.437	+0.003	-0.074
Loan	0.803	-0.044	+0.028	+0.013	-

D. Ablation Analysis

For the sake of simplicity, ablation analysis are only implemented for classification datasets. We focus on conducting an ablation study to analyse the impact of the different components of CTAB-GAN and CTAB-GAN+.

1) With CTAB-GAN: To illustrate the efficiency of each strategy we implement four ablation studies which cut off the different components of CTAB-GAN one by one: (1) **w/o** $\mathcal{C}$. In this experiment, Classifier $\mathcal{C}$ and the corresponding classification loss for Generator $\mathcal{G}$ are taken away from CTAB-GAN; (2) **w/o I. loss** (information loss). In this experiment, we remove information loss from CTAB-GAN; (3) **w/o MSN**. In this case, we substitute the mode specific normalization based on VGM for continuous variables with min-max normalization and use simple one-hot encoding for categorical variables. Here the conditional vector is the same as for CTGAN; (4) **w/o LT** (long tail). In this experiment, long tail treatment is no longer applied. This only affects datasets with long tailed columns, i.e. Credit and Intrusion.

The results are compared with the reference CTAB-GAN implementing all strategies. All experiments are repeated 3 times, and results are evaluated on the same 5 machine learning algorithms introduced in Sec. IV-B1. The test datasets and evaluation flow are the same as shown in Sec. IV-A and Sec. IV-B. Tab. V shows the results in terms of F1-score difference between ablation and CTAB-GAN. Each part of CTAB-GAN has different impacts on different datasets. For instance, **w/o** $\mathcal{C}$ has a negative impact for all datasets except Credit. Since Credit has only 30 continuous variables and one target variable, the semantic check can not be very effective. **w/o information loss** has a positive impact for Loan, but results degenerate for all other datasets. It can even make the model unusable, e.g. for Intrusion. **w/o MSN** performs bad for Covertype, but has little impact for Intrusion. Credit w/o MSN performs better than original CTAB-GAN. This is because out of 30 continuous variables, 28 are nearly single mode Gaussian distributed. The initialized high number of modes, i.e. 10, for each continuous variable (same setting as in CTGAN) degrades the estimation quality. **w/o LT** has the biggest impact on Intrusion, since it contains 2 long tail columns which are important predictors for the target column. For Credit, the influence is limited. Even if the long tail treatment fits well the *amount* column (see Sec. IV-E), this variable is not a strong predictor for the target column.

Table VI: Ablation Analysis For CTAB-GAN+ (F1. diff.)

Dataset	CTAB-GAN+	w/o GT	w/o Was+GP
Adult	0.684	+0.013	-0.029
Covertype	0.636	-0.196	-0.012
Credit	0.802	-0.303	-0.08
Intrusion	0.912	-0.041	-0.049
Loan	0.806	-0.001	+0.003

2) With CTAB-GAN+: To show the efficacy of the General Transform and Was+GP loss in CTAB-GAN+, we propose two ablation studies. (1) **w/o GT** which disables the general transform in CTAB-GAN+. All continuous variables use MSN and all the categorical variables use one-hot encoding. (2) **w/o Was+GP** which switches the default GAN training loss from Was+GP to the original GAN loss defined in [12]. It is worth noting that the information, downstream and generator losses are still present in this experiment. The other experimental settings are the same as in Sec. IV-D1. Tab. VI shows the results in terms of F1-score difference among different versions of CTAB-GAN+. For Covertype, Credit and Intrusion datasets, the effects of GT and Was+GP are all positive. GT significantly boosts the performance on Covertype and Credit datasets. But for Adult, it worsens the result. The reason is that the Adult dataset contains only one GT column: age. Since this column is strongly correlated with other columns, the original MSN encoding can better capture this interdependence. The positive impact of Was+GP on the other hand is limited but consistent across all datasets. The only exception is the Loan dataset, where GT and Was+GP have minor impacts. This is due to the fact that Loan has fewer variables comparing to other datasets, which makes it easier to capture the correlation between columns. CTAB-GAN already performs well on Loan, Therefore, GT and Was+GP cannot further improve performance on this dataset.

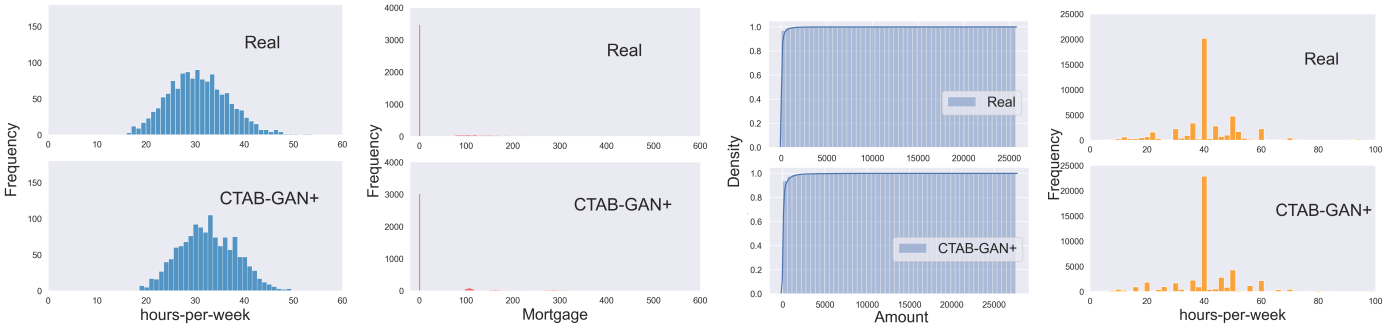
E. Results for Motivation Cases

After reviewing all the metrics, let us recall the four motivation cases from Sec. I-A.

Single Gaussian variables. Fig. 6a(a) shows the real and CTAB-GAN+ generated *bmi* variable. CTAB-GAN+ can reproduce the distribution with minor differences. This shows the effctiveness of general transform to better model variables with single Gaussian distribution.

Mixed data type variables. Fig. 6(b) compares the real and CTAB-GAN+ generated variable *Mortgage* in Loan dataset. CTAB-GAN+ encodes this variable as mixed type. We can see that CTAB-GAN+ generates clear 0 values and the frequency is close to real data.

Long tail distributions. Fig. 6(c) compares the cumulative frequency graph for the *Amount* variable in Credit. This variable is a typical long tail distribution. One can see that CTAB-GAN+ perfectly recovers the real distribution. Due to log-transform data pre-processsing, CTAB-GAN+ learns this structure significantly better than the SOTA methods shown in Fig. 1(c).



(a) *bmi* in Insurrance (b) *Mortgage* in Loan (c) *Amount* in Credit (d) *Hours-per-week* in Adult

Figure 6: Challenges of modeling industrial dataset using existing GAN-based table generator: (a) simple gaussian (b) mixed type, (c) long tail distribution, and (d) skewed data

Skewed multi-mode continuous variables. Fig. 6(d) compares the frequency distribution for the continuous variable *Hours-per-week* from Adult. Except the dominant peak at 40, there are many side peaks. Fig. 1(d), shows that TableGAN, CWGAN and MedGAN struggle since they can learn only a simple Gaussian distribution due to the lack of any special treatment for continuous variables. CTGAN, which also use VGM, can detect other modes. Still, CTGAN is not as good as CTAB-GAN. The reason is that CTGAN lacks the mode of continuous variables in the conditional vector. By incorporating the mode of continuous variables into conditional vector, we can apply the training-by-sample and logarithm frequency also to modes. This gives the mode with less weight more chance to appear in the training and avoids the mode collapse.

V. Experiment Analysis for Differential Privacy

In this section, we show the effect of adding DP to CTAB-GAN+ and compare CTAB-GAN+ with three SOTA DP GAN algorithms.

A. Experiment Setup

Datasets. For sake of simplicity, we only use the classification datasets: Adult, Covertype, Intrusion, Credit and Loan.

Metrics. We use the same ML utility metrics from Section IV-B under two privacy budgets, i.e., $\epsilon = 1$ and $\epsilon = 100$.

Baselines. CTAB-GAN+ is compared against 3 SOTA architectures: PATE-GAN [16], DP-WGAN [32] and GS-WGAN [6]. The code of PATE-GAN and DP-WGAN is taken from Private Data Generation Toolbox[5] which already adapts them for tabular data synthesis. We extend GS-WGAN to the tabular domain by converting each data row into a bitmap image. We first normalize all values to the range $[0, 1]$ and re-shape rows in the form of square images filling missing entries (if any) with zeros. The re-shaped rows are fed into the algorithm and the generated images are transformed into data rows by reversing the previous two operations. All hyper-parameters are kept to their default values except for the default network architecture which is adjusted according to the spatial dimensions of the tabular datasets. Lastly, note that to compute privacy cost fairly, the RDP accountant is used for all approaches that use DP-SGD as it provides tighter privacy guarantees than the moment accountant [31].

Privacy accounting. To compute the privacy cost in a fair manner, we use the RDP accountant for all approaches that employ DP-SGD: CTAB-GAN+, DP-WGAN and GS-WGAN. PATE-GAN uses moment accountant [31] by default. We set $\delta = 10^{-5}$ for all experiments. We follow the examples of DP-WGAN and set the exploration span of λ to $[2, 4096]$. We use (1) to convert the overall cumulative privacy cost computed in terms of RDP back to (ϵ, δ)-DP.

B. Results Analysis

ML Utility Tab. VII presents the results for the differences ML utility between models trained on the original and synthetic data: lower is better. CTAB-GAN+ outperforms all other SOTA algorithms under both privacy budgets. With a looser privacy budget, i.e., higher ϵ, almost all metrics for all algorithms improve. The only exception is AUC for GS-WGAN, but the difference is minor. These results are in line with our expectation because higher privacy budgets mean training the model with less injected noise and more training epochs – before exhaustion of the privacy budget. The superior performance of CTAB-GAN+ compared to other baselines can be explained by its sophisticated neural network architecture, i.e., conditional GAN, which improves the training objective and capacity to better deal with the challenges of the tabular domain such as imbalanced categorical columns and mixed data-types. This also explains the poor results offered by GS-WGAN which is not designed to handling these specific issues achieving the worst overall performance.

Statistical Similarity. Tab. VIII summarizes the statistical similarity results. Among all DP models, CTAB-GAN+ and GS-WGAN consistently improve across all metrics when the privacy budget is increased. But the performance of GS-WGAN is significantly worse than CTAB-GAN+. With a higher privacy budget, Avg WD of PATE-GAN is slightly increased. And the correlation difference of DP-WGAN increases too. This highlights the inability of this methods to capture the statistical distributions during training despite a

[5]https://github.com/BorealisAI/private-data-generation

Table VII: Difference of accuracy (%), F1-score, AUC and AP between original and synthetic data: average over 5 ML models and 5 datasets with different privacy budgets $\epsilon = 1$ & $\epsilon = 100$.

Method	$\epsilon = 1$			$\epsilon = 100$		
	Accuracy	F1-Score	AUC	Accuracy	F1-Score	AUC
CTAB-GAN+	**19.12%**	**0.320**	**0.311**	**13.34%**	**0.311**	**0.299**
PATE-GAN	38.17%	0.513	0.447	37.96%	0.508	0.394
DP-WGAN	36.88%	0.536	0.510	27.27%	0.502	0.423
GS-WGAN	64.10%	0.668	0.492	58.22%	0.639	0.498

Table VIII: Statistical similarity metrics between original and synthetic data: average on 5 datasets with different privacy budgets $\epsilon = 1$ & $\epsilon = 100$.

Method	$\epsilon = 1$			$\epsilon = 100$		
	Avg JSD	Avg WD	Diff. Corr.	Avg JSD	Avg WD	Diff. Corr.
CTAB-GAN+	**0.192**	**775**	**5.61**	**0.137**	**655**	**5.54**
PATE-GAN	0.356	8632	9.45	0.366	8634	8.94
DP-WGAN	0.362	8632	9.19	0.359	8632	9.49
GS-WGAN	0.624	4.07E+06	15.61	0.547	54574	13.74

looser privacy budget. This can be explained by the lack of an effective training framework for dealing with complex statistical distributions present in the tabular domain which arise from imbalances in categorical columns and skews in continuous columns commonly found in real-world tabular datasets.

VI. Conclusion

Motivated by the importance of data sharing and fulfillment of governmental regulations, we propose CTAB-GAN+, a conditional GAN based tabular data generator. CTAB-GAN+ advances beyond SOTA methods by modeling mixed variables and providing strong generation capability for imbalanced categorical variables, and continuous variables with complex distributions. The core features of CTAB-GAN+ include: (1) introduction of the auxiliary component, i.e., classifier or regressor, into conditional GAN, (2) effective data encodings for mixed and simple Guassian variables, (3) a novel construction of conditional vectors and (4) tailored DP discriminator for tabular GAN. We exhaustively evaluate CTAB-GAN+ against state-of-the-art baselines on seven tabular datasets under a wide range of metrics, namely for ML utility and statistical similarity. Results show that the synthetic data of CTAB-GAN+ results into higher ML utility and higher similarity. The overall improvement on classification datasets is at least 56.4% (AUC) and 41.2% (accuracy) compared to related work with no privacy guarantees. When turning on differential privacy, CTAB-GAN+ averagely outperforms at least 48.16% on accuracy and 38.05% on F1-Score than existing DP-GAN training under all privacy budgets. The substantial results of CTAB-GAN+ demonstrate its potential for a wide range of applications that greatly benefit from data sharing, such as banking, insurance, and manufacturing.

References

[1] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[2] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th ICML - Volume 70*, page 214–223. JMLR.org, 2017.

[3] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning (ICML)*, 2017.

[4] M. G. Bellemare, I. Danihelka, W. Dabney, S. Mohamed, B. Lakshminarayanan, S. Hoyer, and R. Munos. The cramer distance as a solution to biased wasserstein gradients. *ArXiv*, abs/1705.10743, 2017.

[5] C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006.

[6] D. Chen, T. Orekondy, and M. Fritz. Gs-wgan: A gradient-sanitized approach for learning differentially private generators. *arXiv preprint arXiv:2006.08265*, 2020.

[7] D. Chen, N. Yu, Y. Zhang, and M. Fritz. Gan-leaks: A taxonomy of membership inference attacks against generative models. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.

[8] E. Choi, S. Biswal, B. Malin, J. Duke, W. F. Stewart, and J. Sun. Generating multi-label discrete patient records using generative adversarial networks. *arXiv preprint arXiv:1703.06490*, 2017.

[9] C. Dwork. Differential privacy: A survey of results. In *International Conference on Theory and Applications of Models of Computation (TAMC)*. Springer, 2008.

[10] C. Dwork, A. Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 2014.

[11] J. Engelmann and S. Lessmann. Conditional wasserstein gan-based oversampling of tabular data for imbalanced learning. *arXiv preprint arXiv:2008.09202*, 2020.

[12] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Proceedings of the 27th NIPS - Volume 2*, page 2672–2680, Cambridge, MA, USA, 2014.

[13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of wasserstein gans. In *the 31st NIPS*, page 5769–5779, 2017.

[14] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, 2017.

[15] M. B. Hawes. Implementing differential privacy: Seven lessons from the 2020 united states census. *Harvard Data Science Review*, 4 2020.

[16] J. Jordon, J. Yoon, and M. van der Schaar. Pate-gan: Generating synthetic data with differential privacy guarantees. In *International Conference on Learning Representations (ICLR)*, 2018.

[17] T. Karras, S. Laine, and T. Aila. A style-based generator architecture for generative adversarial networks. In *IEEE/CVF CVPR*, pages 4396–4405, 2019.

[18] Z. Lin, A. Khetan, G. Fanti, and S. Oh. Pacgan: The power of two samples in generative adversarial networks. *IEEE JSAIT*, 1(1):324–335, 2020.

[19] Y. Long, S. Lin, Z. Yang, C. A. Gunter, and B. Li. Scalable differentially private generative student model via pate. *arXiv preprint arXiv:1906.09338*, 2019.

[20] I. Mironov. Rényi differential privacy. In *Computer Security Foundations Symposium (CSF)*. IEEE, 2017.

[21] A. Mottini, A. Lheritier, and R. Acuna-Agost. Airline Passenger Name Record Generation using Generative Adversarial Networks. In *workshop on Theoretical Foundations and Applications of Deep Generative Models. ICML*, July 2018.

[22] A. Narayanan and V. Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, pages 111–125, 2008.

[23] A. Odena, C. Olah, and J. Shlens. Conditional image synthesis with auxiliary classifier gans. In *The 34th ICML - Volume 70*, page 2642–2651, 2017.

[24] N. Papernot, M. Abadi, U. Erlingsson, I. Goodfellow, and K. Talwar. Semi-supervised knowledge transfer for deep learning from private training data. *arXiv preprint arXiv:1610.05755*, 2016.

[25] N. Park, M. Mohammadi, K. Gorde, S. Jajodia, H. Park, and Y. Kim. Data synthesis based on generative adversarial networks. *Proc. VLDB Endow.*, 11(10):1071–1083, June 2018.

[26] B. Proven-Bessel, Z. Zhao, and L. Chen. Comicgan: Text-to-comic generative adversarial network. *arXiv preprint arXiv:2109.09120*, 2021.

[27] T. Stadler, B. Oprisanu, and C. Troncoso. Synthetic data–a privacy mirage. *arXiv preprint arXiv:2011.07018*, 2020.

[28] A. Torfi, E. A. Fox, and C. K. Reddy. Differentially private synthetic medical data generation using convolutional gans. *arXiv preprint arXiv:2012.11774*, 2020.

[29] R. Torkzadehmahani, P. Kairouz, and B. Paten. Dp-cgan: Differentially private synthetic data and label generation. In *Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2019.

[30] R. Wang, B. Fu, G. Fu, and M. Wang. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*, New York, NY, USA, 2017.

[31] Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan. Subsampled rényi differential privacy and analytical moments accountant. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2019.

[32] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou. Differentially private generative adversarial network. *arXiv preprint arXiv:1802.06739*, 2018.

[33] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni. Modeling tabular data using conditional gan. In *NIPS*, 2019.

[34] A. Yahi, R. Vanguri, and N. Elhadad. Generative adversarial networks for electronic health records: A framework for exploring and evaluating methods for predicting drug-induced laboratory test trajectories. In *NIPS workshop*, 2017.

[35] T. Younesian, Z. Zhao, A. Ghiassi, R. Birke, and L. Y. Chen. Qactor: On-line active learning for noisy labeled stream data. *arXiv preprint arXiv:2001.10399*, 2020.

[36] X. Zhang, S. Ji, and T. Wang. Differentially private releasing via deep generative model (technical report). *arXiv preprint arXiv:1801.01594*, 2018.

[37] Z. Zhao, R. Birke, R. Han, B. Robu, S. Bouchenak, S. B. Mokhtar, and L. Y. Chen. Enhancing robustness of on-line learning models on highly noisy data. *IEEE Transactions on Dependable and Secure Computing*, 18(5):2177–2192, 2021.

[38] Z. Zhao, S. Cerf, R. Birke, B. Robu, S. Bouchenak, S. B. Mokhtar, and L. Y. Chen. Robust anomaly detection on unreliable data. In *49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2019.

[39] Z. Zhao, A. Kunar, R. Birke, and L. Y. Chen. Ctab-gan: Effective table data synthesizing. In *Proceedings of The 13th Asian Conference on Machine Learning*, volume 157, pages 97–112, 17–19 Nov 2021.